# SYMBIOSIS INTERNATIONAL UNIVERSITY, DUBAI

## PRACTICAL ASSESSMENT – LINKED LIST APPLICATION DESIGN

| | |
|---|---|
| **Course Title** | Data Structures |
| **Mode** | Practical – programming |
| **Duration** | 1 hour |
| **Maximum Marks** | 10 |
| **CLO assessed** | CLO 4 – Implement various types of linked lists (singly, doubly, and circular) based on structure, operations, and use cases |

### Instructions to candidates

1. Attempt **all** parts. Marks for each part are shown in brackets. The whole assessment is mapped to **CLO 4**.
2. Write your program in the language used in the course (C, C++, Java or Python). The playlist must be stored in a **linked list that you build yourself**. Do not use arrays or built-in list types (such as vector, ArrayList or Python lists) to hold the songs.
3. Save your program as **RollNo_Name_Playlist** with the correct file extension.
4. Answer Parts A and D on this paper. Submit this paper, your source file and a screenshot or printout of the program output.
5. A program that does not run completely can still earn partial marks, so complete and save each function in turn.

## Scenario: Melody Café Music Player

A café plays music through a small player program. The playlist changes during the day: songs are added at the end or inserted after a particular song, songs are removed when customers complain, and the staff move forward and backward through the songs. When the last song finishes, the player repeats the playlist from the first song. Each song has a **title** (up to 30 characters) and a **duration in seconds**.

| Operation | Required behaviour |
|---|---|
| addSong(title, duration) | Adds a new song at the end of the playlist. |
| insertAfter(key, title, duration) | Inserts a new song immediately after the song named key. Prints a message if key is not found. |
| removeSong(title) | Removes the first song with this title, whether it is the first, a middle or the last song. Prints a message if it is not found. |
| displayForward() | Prints the titles from the first song to the last song. |
| displayBackward() | Prints the titles from the last song to the first song. |
| playNext(current) | Returns the song after current. After the last song it returns the first song. |

### Part A. Design (1.5 Marks)

a. Explain why a linked list is a better choice than an array for this use case. **(0.5 mark)**

b. Choose the type of linked list (singly, doubly or circular) that best suits the player, and justify your choice using the required operations. **(0.5 mark)**

c. Write the node structure (or class) for a song. **(0.5 mark)**

### Part B. Implementation (5.5 Marks)

Implement the six operations in the table above. Your code must work correctly for an **empty playlist**, the **first** song, the **last** song, a **middle** song and a **song that is not found**.

a. addSong **(1 mark)**

b. insertAfter **(1.5 marks)**

c. removeSong **(1.5 marks)**

d. displayForward and displayBackward **(1 mark)**

e. playNext **(0.5 mark)**

### Part C. Testing (2 Marks)

Write a main program that performs the following test and prints the results:

1. Add the songs: Sunrise (210), Desert Rain (185), Marina Lights (240).
2. Insert Skyline (200) after Sunrise.
3. Insert Night Drive (195) after Marina Lights.
4. Display the playlist forward and backward.
5. Remove Sunrise, then Desert Rain, then Night Drive, then Unknown.
6. Display the playlist forward and backward again.
7. Starting from Marina Lights, call playNext and print the title of the song returned.

Submit a screenshot or printout of your program output.

### Part D. Types of Linked Lists and Use Cases (1 Mark)

a. The café also keeps a separate **song-request queue**. Requests are added at the end and served from the front, and the list is never read backward. State the type of linked list that best suits this queue and explain why. **(0.5 mark)**

b. Suppose the playlist were a **doubly circular** linked list. State which pointers link the two ends, and describe two changes this would make to your functions. **(0.5 mark)**

## Suggested time plan and mark summary

| Part | CLO | Suggested time | Marks |
|---|---|---|---|
| A. Design | CLO 4 | 8 minutes | 1.5 |
| B. Implementation | CLO 4 | 35 minutes | 5.5 |
| C. Testing | CLO 4 | 10 minutes | 2 |
| D. Types and use cases; save and submit | CLO 4 | 7 minutes | 1 |
| **Total** | | **60 minutes** | **10** |

*— End of Question Paper —*